SESION N° 3

1. Cambios de estructura sin apagar el servidor

-- Sesión 3: Cambios de estructura sin interrumpir el servicio ("evento de lanzamiento")

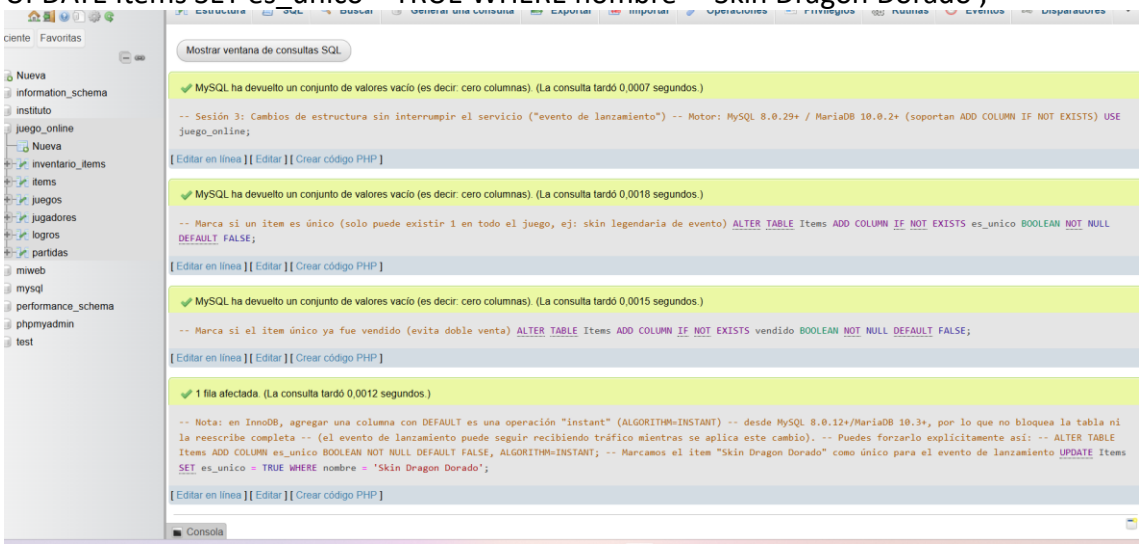
-- Motor: MySQL 8.0.29+ / MariaDB 10.0.2+ (soportan ADD COLUMN IF NOT EXISTS)
USE juego_online;

-- Marca si un item es único (solo puede existir 1 en todo el juego, ej: skin legendaria de evento)
ALTER TABLE Items
  ADD COLUMN IF NOT EXISTS es_unico BOOLEAN NOT NULL DEFAULT FALSE;

-- Marca si el item único ya fue vendido (evita doble venta)
ALTER TABLE Items
  ADD COLUMN IF NOT EXISTS vendido BOOLEAN NOT NULL DEFAULT FALSE;

-- Nota: en InnoDB, agregar una columna con DEFAULT es una operación "instant" (ALGORITHM=INSTANT)
-- desde MySQL 8.0.12+/MariaDB 10.3+, por lo que no bloquea la tabla ni la reescribe completa
-- (el evento de lanzamiento puede seguir recibiendo tráfico mientras se aplica este cambio).
-- Puedes forzarlo explícitamente así:
-- ALTER TABLE Items ADD COLUMN es_unico BOOLEAN NOT NULL DEFAULT FALSE, ALGORITHM=INSTANT;

-- Marcamos el item "Skin Dragon Dorado" como único para el evento de lanzamiento
UPDATE Items SET es_unico = TRUE WHERE nombre = 'Skin Dragon Dorado';



Explicación:
Este código sirve para preparar la tienda ante un evento importante o vender un objeto súper exclusivo que solo una persona en todo el mundo puede tener
Lo mejor de este código es que hace los cambios mientras el juego sigue funcionando, sin necesidad de apagar los servidores ni sacar a los jugadores de sus partidas.
Este código sirve para actualizar la tienda de un videojuego en vivo de forma segura.

2. Procedimiento con bloqueo de filas

```
USE juego_online;

DROP PROCEDURE IF EXISTS comprar_item_unico;

DELIMITER //

CREATE PROCEDURE comprar_item_unico(
   IN p_jugador_id INT,
   IN p_item_id INT,
   OUT p_resultado VARCHAR(255)
)
BEGIN
   DECLARE v_vendido  BOOLEAN;
   DECLARE v_precio   INT;
   DECLARE v_monedas  INT;

   START TRANSACTION;

   -- FOR UPDATE bloquea la fila hasta que esta transacción termine
(COMMIT/ROLLBACK),
   -- evitando que dos compras concurrentes lean "vendido = false" al mismo tiempo.
   SELECT vendido, precio_monedas INTO v_vendido, v_precio
   FROM Items
   WHERE item_id = p_item_id AND es_unico = TRUE
   FOR UPDATE;

   IF v_vendido THEN
      SET p_resultado = 'RECHAZADO: el item único ya fue vendido a otro jugador.';
      ROLLBACK;
   ELSE
      SELECT monedas INTO v_monedas FROM Jugadores WHERE jugador_id =
p_jugador_id;

      IF v_monedas < v_precio THEN
         SET p_resultado = 'RECHAZADO: monedas insuficientes.';
         ROLLBACK;
      ELSE
         UPDATE Items SET vendido = TRUE WHERE item_id = p_item_id;
         UPDATE Jugadores SET monedas = monedas - v_precio WHERE jugador_id =
p_jugador_id;
         INSERT INTO Inventario_Items (jugador_id, item_id, cantidad)
            VALUES (p_jugador_id, p_item_id, 1)
            ON DUPLICATE KEY UPDATE cantidad = cantidad + 1;

         SET p_resultado = CONCAT('EXITO: item unico vendido al jugador ',
p_jugador_id);
         COMMIT;
      END IF;
   END IF;
END
```



Explicacion:
Este código sirve para que el juego sea justo y seguro. Utiliza "candados digitales" (FOR UPDATE) para atender a los jugadores uno por uno cuando intentan comprar lo mismo al mismo tiempo. Así se evita que el juego se confunda, que robe dinero o que clone un objeto único por error.

3. Simular el "Evento de Lanzamiento"

Ahora vamos a reproducir el escenario real: dos jugadores distintos intentando comprar el mismo item único casi al mismo instante, usando las dos pestañas de phpMyAdmin

Pestaña A · Jugador 2 (ShadowNinja)

CALL comprar_item_unico(2, 1, @resultado_a);
SELECT @resultado_a AS resultado;

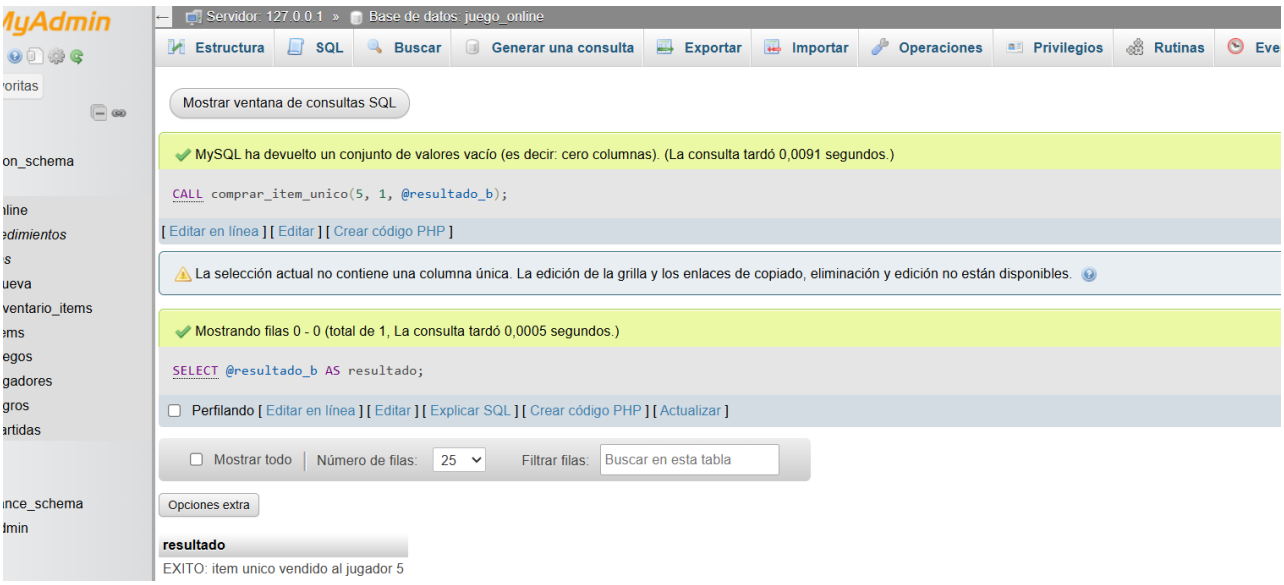


Explicacion:

Esto activa el procedimiento que creamos. Es el momento exacto en el que un jugador presiona el botón "Comprar" en el juego.
Luego mostramos en la pantalla del administrador el mensaje que el código anotó dentro de la caja @resultado_a.

Pestaña B · Jugador 5 (RocketFox)

CALL comprar_item_unico(5, 1, @resultado_b);
SELECT @resultado_b AS resultado;



Explicacion:
Aquí envía la orden de compra del segundo jugador en discordia.
Luego mostramos en la pantalla qué pasó con la billetera.
Esto demuestra que es matemáticamente imposible que el objeto se clone o se venda dos veces, validando que tu código de base de datos es 100% seguro para un entorno real con miles de jugadores.